



Lecture 8: Bash fancy features

Last update: 20250711

Table of Contents

1. [Subshells: \(... \)](#)
2. [Process substitution operator: <\(... \)](#)
3. [Here strings: <<<](#)
4. [Here documents: <<](#)
5. [Menus](#)

1. Subshells: (...)

We have already seen how the code block `{ ... }` can be used in **Bash** to embed the specific code snippet inside and allocate only to the execution of that code snippet the separate I/O facilities from the rest of the code. Another way of achieving this functionality is to use *subshell* `( ... )`. Any code block, when embedded within the round braces `( ... )`, forms the subshell. Generically, a subshell is defined as:

```
(  
  command-input-1  
  command-input-2  
  ...  
  command-input-n  
)
```

or completely equivalently, with a one-liner:

```
( command-input-1; command-input-2; ... ; command-input-n; )
```

Just like it was done for the code block `{ ... }` in the previous sections, we now provide an executive summary of the most important features of the subshell:

1. `( ... )` inherits the current environment and cannot modify it globally;
2. `( ... )` has its own `1>` and `2>` streaming facilities;
3. `( ... )` starts a separate process, its PID can be obtained programmatically from the built-in variable **BASHPID**.

From the above list, it is clear that there are a lot of similarities between `{ ... }` and `( ... )`. Basically, there are only two important differences:

1. when compared to the parent shell, `( ... )` starts a new process, while `{ ... }` does not;
2. both `( ... )` and `{ ... }` inherit the environment from the parent shell, but `{ ... }` can modify it globally while `( ... )` cannot. For a subshell `( ... )`, the fact that it cannot modify the global environment remains true even if the keyword **export** has been used in declarations in the subshell.

When it comes to *stdout* and *stderr* streams, there is no difference between `( ... )` and `{ ... }`. The subshell `( ... )` is usually less efficient than the code block `{ ... }` because it runs a separate process and time has to be invested in starting a new process. However, since it cannot modify the environment in which it is run globally, `( ... )` is safer. As a rule of thumb, `( ... )` shall be preferred over `{ ... }` unless efficiency is a concern.

Typically, subshells are executed in the background, by using the following generic syntax:

```
( command-input-1; command-input-2; ... ; command-input-n; ) &
```

The advantage of running subshells in the background is that now, by using the **wait** command, we can decide whether the rest of the code in the script will or will not wait for the subshell execution to terminate (just like for any other command running in the background in its own process). This is very handy because we can use in that subshell automatically the already initialized environment in the script, execute the subshell, use its result, and keep the environment unmodified.

As a side remark, we clarify the difference between the content of **\$BASHPID** and **\$\$**. In the current shell, they both expand to the process ID of that shell:

```
$ echo $$; echo $BASHPID;
1217831
1217831
```

However, in a subshell, only **BASHPID** gets reset to the new process ID of a subshell:

```
$ ( echo $$; echo $BASHPID; )
1217831
1217887
```

Therefore, by using them both in a subshell, we can trace back separately the PID of a parent shell from which a subshell was started, and of that subshell.

The use case of the subshell is illustrated with the following simple example:

```
( echo "Subshell PID: ${BASHPID}"; date; date -q; sleep 10m; ) 1>output.log 2>error.log &
```

Completely equivalently, the above code snippet could have been implemented across multiple lines:

```
(
  echo "Subshell PID: ${BASHPID}"
  date
  date -q
  sleep 10m
) 1>output.log 2>error.log &
```

After executing, we see with **jobs -l** the following printout:

```
[2]+  7941 Running                  ( echo "Subshell PID: ${BASHPID}"; date; date -q; sleep 10m ) > output.log 2> error.log &
```

This means that the whole composite code inside the subshell now behaves like any other command running in the background. Any *stdout* printout from any command in the body of the subshell is redirected with **1>** in the file 'output.log', whose content is:

```
Subshell PID: 7941
Do  4. Jul 07:37:55 CEST 2019
```

On the other hand, any *stderr* stream from any command in the body of the subshell was redirected with **2>** in the separate file 'error.log':

```
date: invalid option -- 'q'
Try 'date --help' for more information.
```

We can suspend the subshell execution just as we did it for individual commands:

```
kill -TSTP 7941
jobs -l
# [2]+  Stopped                  ( echo "Subshell PID: ${BASHPID}"; date; date -q; sleep 10m ) > output.log 2> error.log
```

and resume it:

```
kill -CONT 7941
jobs -l
# [2]+  7941 Running              ( echo "Subshell PID: ${BASHPID}"; date; date -q; sleep 10m ) > output.log 2> error.log &
```

In the same fashion, we can terminate only the subshell execution, without affecting the script execution from which the subshell was launched. This is true whether or not the subshell is executed in the background. For instance, we can have the following schematic implementation in the script:

```
some code
( some massive computation in subshell )
the remaining code waiting subshell to terminate
```

If the execution of the above script gets stuck because of some massive computation in the subshell, instead of terminating the whole script and restarting, we can terminate differentially only the subshell from a separate terminal, by sending some of the signals **TSTP**, **INT**, **QUIT** or **KILL** directly to the PID corresponding to the subshell. This is not possible for code block **{ ... }** because its PID is the same as the PID of the parent shell.

2. Process substitution operator: `<( ... )`

Process substitution operator `<( ... )` is a feature that is not provided by all shells. Therefore, its usage typically leads to some portability problems across different shells. Nevertheless, since it is supported by **Bash**, and since it comes in very handy in some frequently encountered cases in practice, we introduce it next.

Loosely speaking, the process substitution operator translates on the fly the standard output stream of any command into a 'virtual' file, which then can be fed to the commands that accept only files as arguments. The true mechanism behind the scene is much more complicated than that (it relies on so-called 'named pipes', see the man pages of the **mkfifo** command), but this loose explanation captures its behavior in practice quite well.

Within the `<( ... )` operator, we can execute as many commands as we wish, separated with `;` delimiter. Because of this, the process substitution operator offers more flexibility than pipe `|`, which can forward directly the *stdout* of only one command as an input to another command.

In we execute multiple commands within the `<( ... )` operator, the output of each command is summed up in one large common 'virtual' file, which then we can feed for further processing to commands like **grep**, **awk**, etc. This is best illustrated with the examples.

Example 1: How to extract only the text in the 4th column from the output of three consecutive commands? By using the process substitution operator and **awk**, the solution is very simple and elegant:

```
awk '{print $4}' <(command1; command2; command3)
```

In order to solve this problem without using the `<( ... )` operator, we would need to use something like:

```
command1 > someFile
command2 >> someFile
command3 >> someFile
awk '{print $4}' someFile
rm someFile
```

Clearly, by using the process substitution operator `<( ... )` we have saved quite some unnecessary coding steps.

Example 2: How to parse line-by-line through the output of some command and manipulate each line programmatically?

We have already seen that by using the **while+read** construct we can parse line-by-line through the content of some external file. By using the process substitution operator `<( ... )`, we can trivially extend that functionality to the output of some command. For instance, let us do some programmatic manipulation on the output of **ls -al** command:

```
while read; do
  echo "In line $REPLY there are ${#REPLY} characters"
done <(ls -al)
```

The output of this command could look like this:

```
In line -rw-r--r--  1 abilandz abilandz  9508 Jul  2 09:42 bash_logo.png there are 64
characters
In line -rw-rw-r--  1 abilandz abilandz  7652 Jul  2 10:51 colours.png there are 62
characters
```

This works because from the perspective of the **while+read** construct, redirection from an external file via `< someFile` or via process substitution operator `< <( ... )` is completely equivalent because the output of `<( ... )` is essentially a 'virtual' file.

Example 3: How to check programmatically if the printouts of two commands are exactly the same?

In order to compare the contents of two files, we can use the **diff** command, with the following generic syntax:

```
diff file1 file2 && echo same || echo different
```

With the process substitution operator, we can extend the functionality of the **diff** command to compare the printouts of different commands. The generic use case could look like:

```
diff <(command1) <(command2) && echo same || echo different
```

Without the process substitution operator, we would need to dump the printout of each command in temporary files and then compare the content of those temporary files with the **diff** command. Clearly, the process substitution operator also saves, in this frequently encountered example, a lot of additional and completely redundant coding.

Example 4: How to parse over files or directories which have empty characters in their name?

```
# As a starting point, we consider the case when in a directory named TEMP
# there are two subdirectores, named "dir 1" and "dir_2"

$ mkdir TEMP
$ mkdir 'TEMP/dir 1' 'TEMP/dir_2'
$ ls TEMP
'dir 1'  dir_2

# Wrong solution with 'for' loop and command substitution operator:
$ for subdir in $(find "TEMP" -mindepth 1 -type d); do echo $subdir; done
TEMP/dir_2
TEMP/dir
1

# Correct solution with 'while' loop and process substitution operator:
$ while read subdir; do echo $subdir; done < <(find "TEMP" -mindepth 1 -type d)
TEMP/dir_2
TEMP/dir 1
```

In the above example, we have used the command **find**, which is covered in detail in Lecture 9.

3. Here strings: <<<

'Here strings' is a **Bash** feature that can save a bit of typesetting in some frequently encountered use cases. For instance, with 'here strings', we can feed the already-made string directly into the command without typing that string manually. For some commands (e.g. **bc**), we first need to start, and only then supply the string as an input interactively from the keyboard. Such cases can also be bypassed with 'here strings' and instead of typing directly from the keyboard, the commands can process the input string programmatically.

The generic syntax for the usage of 'here strings' can be represented in the following schematic way:

```
command <<< "some hardwired string"
```

In the above example, the input string is hardwired, but it can also be supplied as the content of the variable:

```
Var="some hardwired string"
command <<< ${Var}
```

The command substitution operator `$( ... )` and arithmetic expansion `$(( ... ))` can be used as well on the right-hand side of `<<<` operator. What **Bash** is doing when it encounters `<<<` can be summarized as follows: Whatever is on the right-hand side of `<<<` undergoes expansion (e.g. `${Var}` is replaced with variable content, `$(( ... ))` is replaced with the result of arithmetic evaluation, etc.), and then the resulting expression is simply fed to the *stdin* of **command**. The result of the expansion is supplied as a single string to **command**, with a newline always appended.

It is also possible to think about 'here strings' as being the shortcut notation for the following common construct:

```
echo "someString" | command
```

because the output of the above line is the same as of:

```
command <<< "someString"
```

Therefore, instead of using the command **echo** in combination with pipe `|`, we can simply condense the syntax and only use `<<<` — the final printout is exactly the same.

Example 1: Let's define `Var=20180524`. How to check programmatically with **grep** if this string begins with the pattern '2018'?

The two solutions below yield the same result:

```
$ Var=20180524
$ echo ${Var} | grep "^2018"
20180524
$ grep "^2018" <<< ${Var}
20180524
```

'Here strings' are frequently combined with **sed**, and that is illustrated in the next example.

Example 2: Let's define `Var=20180524`. How to change the pattern '2018' into '2020' programmatically and immediately redefine the variable `Var` to the new content?

```
$ Var=20180524
$ sed "s/2018/2020/" <<< $Var
20200524
$ echo ${Var}
20180524 # the content of 'Var' is unchanged
```

Note that the content of the starting variable `Var` is still '20180524', we only saw the pattern replacement in the printout on the screen. If we want to update immediately the content of the starting variable, this can be achieved with:

```
$ Var=20180524
$ Var=$(sed "s/2018/2020/" <<< $Var)
$ echo ${Var}
20200524 # the content of 'Var' has changed
```

Finally, we illustrate the typical usage of 'here strings' in combination with **bc** command, to perform floating point arithmetic. We start by recalling the example from Lecture 6.

Example 3 (revised from Lecture 6): How can we divide 10/7 at the precision of 30 significant digits? The solution was given previously by the following code snippet:

```
$ echo "scale=30; 10/7" | bc
1.428571428571428571428571428571
```

However, this problem can also be solved a bit simpler with the usage of 'here strings':

```
$ bc <<< "scale=30; 10/7"
1.428571428571428571428571428571
```

There are cases in which **echo** + `|` combination is more efficient, while there are also cases in which the 'here strings' `<<<` run faster, so both versions are used equally frequently in practice.

4. Here documents: `<<`

'Here documents' are typically used within shell scripts to write programmatically entire files, including the new scripts. Programmatically written files with 'here documents' can be used as a further input to the script which made them or as an input to some other commands. Programmatically written scripts with 'here documents' can be immediately executed in the script that created them.

There are three important use cases of 'here documents'. The first one uses the following generic syntax:

```
cat > someFile <<HERE-DOC
... some code here ...
HERE-DOC
```

The 'here document' begins with `<<` and its body is marked with the matching pair of delimiters. The name of the delimiters is arbitrary, as long as the closing one does not coincide with some string in the code in the body. The above construct evaluates all code inside the matching pair of delimiters named 'HERE-DOC', and dumps it in the external file named 'someFile'. There is nothing special about the choice of 'HERE-DOC' to name the delimiters, we could have used as well something like

```
cat > someFile <<EOF
... some code here ...
EOF
```

The empty characters at the end of delimiters do matter, and this is a typical source of errors. If the name of the opening delimiter is 'HERE-DOC' (no empty characters at the end!) and the name of the closing delimiter is 'HERE_DOC ' (one empty character at the end!), we will get an error, as the two delimiters do not match each other exactly. Therefore, it is mandatory to ensure that there are no trailing empty characters after the delimiters.

If the above version of 'here documents' is used, **Bash** supports variable and command substitution in the body of 'here documents', which enables to write files programmatically. This is best illustrated with the following simple script named 'testHD.sh', which takes as two arguments two floating-point numbers, sums them up, and dumps everything (alongside with some other info) in the external file named 'output.log':

```
#!/bin/bash

[[ 2 -eq $# ]] || return 1

cat > output.log <<HERE-DOC
Today is: $(date)
File produced by script: ${BASH_SOURCE}
Sum is: $(bc <<< "scale=2; $1 + $2")
HERE-DOC

return 0
```

If we execute this script for instance with:

```
source testHD.sh 2.44 10.23
```

this script has by using the 'here document' created an external file named 'output.log', whose content is:

```
Today is: Tue Jun 16 14:04:52 CEST 2020
File produced by script: testHD.sh
Sum is: 12.67
```

Note that we did not need to use **echo** in the body of 'here documents' to dump any text into the file 'output.log'. On the other hand, if we would have used the code block `{ ... }` to achieve the same results, the code inside the code block would be clogged with **echo** statements:


```
#!/bin/bash

[[ 2 -eq $# ]] || return 1

{
  echo Today is: $(date)
  echo File produced by script: ${BASH_SOURCE}
  echo Sum is: $(bc <<< "scale=2; $1 + $2")
} > output.log

return 0
```

Because of this difference, 'here documents' are simpler and more elegant when writing programmatically the files than the code blocks `{ ... }`.

The second important use case of 'here documents' is described with the following generic syntax:

```
cat > someFile.log <<'HERE-DOC'
... some code here ...
HERE-DOC
```

or equivalently with:

```
cat > someFile.log <<"HERE-DOC"
... some code here ...
HERE-DOC
```

Note the use of quotes around the opening delimiter. In this version, the code in the body of 'here documents' is literally evaluated with the following phrase: *What you typed is what you get*. The meaning of all special symbols in the code in the body of 'here document' is killed if the starting delimiter is enclosed within quotes. For instance, if we modify the previous example:

```
#!/bin/bash

[[ 2 -eq $# ]] || return 1

cat > newScript.sh <<'HERE-DOC'
Today is: $(date)
File produced by script: ${BASH_SOURCE}
Sum is: $(bc <<< "scale=2; $1 + $2")
HERE-DOC

return 0
```

and execute it as before with

```
source testHD.sh 2.44 10.23
```

the content of the file 'newScript.sh', to which we have redirected now the content of 'here document', is different:

```
Today is: $(date)
File produced by script: ${BASH_SOURCE}
Sum is: $(bc <<< "scale=2; $1 + $2")
```

This is a perfectly valid **Bash** source code. Literally, what we type in the body of this version of 'here document' is what we get in the external file.

The above version of 'here documents' has an obvious and important use case: We can, within one **Bash** script, programmatically write (and execute immediately if necessary) another **Bash** script, with all special characters and **Bash** syntax in place. This feature is more general because with this version of 'here documents' we can preserve the special syntax of any other programming language, dump the code in an external file, compile it, and use the executable immediately in the very same **Bash** script in which you have written the initial source code.

Finally, 'here documents' can be used to comment out multiple lines of **Bash** source code in one go, instead of using `#` at the beginning of each line. This is illustrated with the following example:

```
... code line 1 ...
... code line 2 ...
... code line 3 ...
... code line 4 ...
... code line 5 ...
... code line 6 ...
... code line 7 ...
... code line 8 ...
... code line 9 ...
```

How for instance we would comment out the code in the lines from 2 to 8, and execute only the code in the lines 1 and 9? This can be achieved in the following way:

```
... code line 1 ...
: <<'HERE-DOC'
... code line 2 ...
... code line 3 ...
... code line 4 ...
... code line 5 ...
... code line 6 ...
... code line 7 ...
... code line 8 ...
HERE-DOC
... code line 9 ...
```

We have essentially declared lines from 2 to 8 to be the body of 'here document', and then its content redirected to the 'do-nothing' command `:` — this is yet another very neat use case of 'do-nothing' command! When using 'here documents' to comment out a piece of code, the version in which the opening delimiter is enclosed in quotes is safer because it prevents any potential variable or command substitution in the body before passing the code over to the 'do-nothing' command.

As this is a frequent source of painful debugging, we close this section by remarking again that the closing delimiter in 'here documents' shall not be followed by any trailing empty character.

5. Menus

Menus are built in **Bash** programmatically with the built-in command **select**, with the following generic syntax:

```
select someVariable in someList; do
    ... some commands ...
break
done
```

The menu for selection is specified via the list, schematically indicated as 'someList', which is placed between the keyword **in** and semicolon `;`. The list can be specified by hardwiring some entries separated with empty characters, using the command substitution operator `$( ... )`, brace expansion mechanism, etc. If the keyword **break** is omitted, the menu is offered again and again for selection.

We illustrate now with a few concrete examples how to build menus in **Bash**. We start by saving the following code in the file 'selectCountry.sh'

```
#!/bin/bash

select Country in Germany Italy France Spain England; do
    echo "You have selected: $Country"
    break
done

return 0
```

After executing the script via:

```
source selectCountry.sh
```

the following menu appears on the screen:

```
1) Germany
2) Italy
3) France
4) Spain
5) England
#?
```

Bash has offered us the predefined menu and is now waiting for a reply. If we for instance type `1` and press `Enter` we get as a result the following printout:

```
You have selected: Germany
```

Since we have used the keyword **break**, the **select** command bails out immediately after the first selection is done.

As another example, we can build a menu from the output of some command:

```
#!/bin/bash

select File in $(ls *.sh); do
    echo "You have selected script: $File"
    source $File
    break
done

return 0
```

The above code offers on the menu all scripts in the current working directory, and then we can directly execute the script from the menu, literally with one key stroke, instead of typing **source scriptName**.

The default prompt symbol in **select** is `#?`, while the default prompt symbol in **Bash** is `$`. How the prompt will look is determined from the content of special environment variables **PS1** (for **Bash**), and **PS3** (for **select**). For instance, we can re-execute the previous example with:

```
PS3="What is your choice? " source selectCountry.sh
```

The menu is now:

```
1) Germany
2) Italy
3) France
4) Spain
5) England
What is your choice?
```

As a concluding remark, we indicate that if the keyword **in** is omitted and the list is not specified, the list is defaulted to the positional parameters supplied to the script or function, and the menu is built out of all supplied positional parameters.